```plantuml
@startuml
class Jeux {
  + Chrono: Chrono
  + Joueur: Joueur
  + Victoire: bool
  + Gestion: GestionScore
  + Interaction: GestionInteraction
  + Quitter: bool
  + ModeTest: bool
  + Jeux()
  + LancerJeu(): void
  + BoucleJeu(): void
  + VerifierVictoire(): void
  + AfficherFin(motif: string): void
}
note right of Jeux : Classe centrale qui gère la boucle du jeu et l'état global

class "<<static>>\nMenuPrincipal" {
  + {static} AffichageMenuJeu(): void
  + {static} AffichageMenuSelection(): string
}

class "<<static>>\nCreationJoueur" {
  + {static} DefinirJoueur(j: Joueur): bool
  + {static} ChoisirEtage(j: Joueur, question: string): int
}

class "<<static>>\nInterfaceGraphiqueJeu" {
  + {static} InitialiserLayout(): Layout
  + {static} AffichageJeu(layout: Layout, joueur: Joueur): void
  + {static} MiseAjourLayout(...): void
}

Jeux --> GestionInteraction : utilise
Jeux --> Chrono : utilise
Jeux --> GestionScore : utilise
Jeux --> Joueur : gère

"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageJeu" : appelle
"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageHeader" : appelle
"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageLegende" : appelle
"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageAide" : appelle
"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageCarte" : appelle
"<<static>>\nInterfaceGraphiqueJeu" --> "<<static>>\nAffichageInteraction" : appelle

class "<<static>>\nAffichageJeu" {
  + {static} ChiffreVersCaractere(chiffre: int): string
  + {static} AfficherMatriceJeu(matrice: List<List<int>>): void
}

class "<<static>>\nAffichageHeader" {
  + {static} AffichageJoueur(joueur: Joueur): void
  + {static} AffichageTemps(temps: string): void
  + {static} AffichageEtage(etage: string): void
}

class "<<static>>\nAffichageLegende" {
  + {static} AffichageLegende(): void
}

class "<<static>>\nAffichageAide" {
  + {static} AffichageAide(): void
}

class "<<static>>\nAffichageCarte" {
  + {static} AffichageCartePetiteEchelle(joueur: Joueur): void
}

class "<<static>>\nAffichageInteraction" {
  + {static} AffichageInteractionJoueur(interaction: int): void
}

class GestionInteraction {
  + TeleportationEchouee: bool
  + XInteraction: int
  + YInteraction: int
  + GestionInteraction(joueur: Joueur)
  + VerifierInteractionJeu(joueur: Joueur): int
  + GererTeleportation(joueur: Joueur): void
  + GererRecuperationCle(joueur: Joueur): void
  + GererValidationPilotage(joueur: Joueur): void
}

GestionInteraction --> Joueur : vérifie

class Chrono {
  + Stopwatch: Stopwatch
  + EnCours: bool
  + TempsTotal: int
  + TempsRestant: int
  + Chrono()
  + LancerChrono(): void
  + AffichageChrono(): string
  + ArreterChrono(): void
  + ComparerChrono(autreTemps: int): bool
}

class GestionScore {
  + CheminFichier: string
  + GestionScore(modeTest: bool)
  + EnregistrerScore(joueur: Joueur, chrono: int): void
  + LireScore(): void
  + AfficherClassement(affichage: bool): void
  {inner} EntreeScore
}

GestionScore --> "<<inner>>\nEntreeScore" : contient

class "<<inner>>\nEntreeScore" {
  + Nom: string
  + Temps: int
}

class Joueur {
  + Nom: string
  + X: int
  + Y: int
  + PositionEtage: int
  + EtageVisite: List<int>
  + ALaCle: bool
  + Vaisseau: Vaisseau
  + Random: Random
  + NomEtageCourant: string
  + EtageCourant: Etage?
  + ABienMisLaCle: bool
  + ModeTest: bool
  + Joueur(modeTest: bool)
  + InitialiserPosition(): void
  + DeplacementDansModule(key: ConsoleKey): void
  + DeplacementModule(key: ConsoleKey): void
  + ChangementEtage(etageNumber: int): void
  + RecupererCle(): void
}
note right of Joueur : Classe principale représentant le joueur et sa position dans le vaisseau

Joueur --> Vaisseau : utilise

class Vaisseau {
  + ListeEtages: List<Etage>
  + ListeEtagesPotentiels: List<string>
  + Random: Random
  + XCle: int
  + YCle: int
  + EtageCleCache: Etage?
  + Cachettes: List<int>
  + Vaisseau(modeTest: bool)
  + CacherLaCle(): void
  + RemplirListeEtage(nombreEtages: int): void
  + AjoutModuleDePilotagessuffisants(): void
  + MetUnModulePilotageValideAleatoire(): void
}

Joueur --> Etage : EtageCourant
Vaisseau o--> Etage : contient
Vaisseau o--> PosteDePilotage : initialise
Vaisseau o--> ModuleTeleportationArrivee : initialise

abstract class Etage {
  - Nom: string
  - ListeModulePotentiels: List<Module>
  - ListeModules: List<Module>
  - MatriceEtage: List<List<int>>
  - Random: Random
  + CompleterAvecModulesPotentiels(): void
  + CompleterListeModuleEspace(): void
  + CompleterAvecEspaces(): List<Module>
  + CreationMatriceEtage(): void
}
note right of Etage : Classe abstraite mère des différents types d'étages du vaisseau

Etage "1" --> "1" Vaisseau : contient
Etage --> "*" Module : contient (ListeModules)
Etage --> Module : possibilité (ListeModulePotentiels)

class SalleMachine {
  + SalleMachine()
}

class LieuDeVie {
  + LieuDeVie()
}

SalleMachine --|> Etage
LieuDeVie --|> Etage

abstract class Module {
  # MatriceModule: List<List<int>>
  + Module()
}
note right of Module : Classe abstraite mère de tous les modules pouvant être placés dans les étages

class PosteDePilotage {
  + PosteDePilotage()
}

class ModuleTeleportationArrivee {
  + ModuleTeleportationArrivee()
}

class ModulePilotage {
  + EstValide: bool
  + ModulePilotage()
  + ModulePilotage(estValide: bool)
  + InitialiserMatrice(): void
}

class Espace {
  + Espace()
  + RemplirTotalement(chiffre: int): void
  + MurVertical(colonne: int): void
  + CoinHautGauche(): void
  + CoinHautDroite(): void
}

class ModuleMoteur {
  + ModuleMoteur()
}

class ModuleCochage {
  + ModuleCouchage()
}

class ModuleCoffre {
  + ModuleCoffre()
}

class ModuleTeleportationDepart {
  + ModuleTeleportationDepart()
}

Module --|> PosteDePilotage
Module --|> ModuleTeleportationArrivee
Module --|> ModulePilotage
Module --|> Espace
Module --|> ModuleMoteur
Module --|> ModuleCochage
Module --|> ModuleCoffre
Module --|> ModuleTeleportationDepart
@enduml
```